

Dunes Insolites

Full-Stack Plan

Backend, espace client, espace admin — and whether two companies can share one platform

You asked whether Route Insolite and Dunes Insolites can share one user table. They already do — one Postgres database, one `users` table, one Keycloak realm, one reservation flow and one product catalogue. The question is not whether it is feasible. It is whether the current model is *complete enough*, and it is not.

Company identity exists in exactly one place — a flag on the invoice — while two separate legal entities share a single invoice number sequence. That is an accounting-integrity problem, not a design preference.

Prepared for	Abdou Hanafi
Date	25 August 2026
Backend	Spring Boot 4.0.3 · Java 21 · Postgres · Keycloak · RabbitMQ · ~12,500 LOC
Espace client	Next.js 16.3.1 · React 19 · Tailwind v4 · ~6,000 LOC
Espace admin	New Next.js app — retires the Angular <code>admin-app</code>
Legal entities	DUNES INSOLITES (Kébili) · ROUTE INSOLITE (Djerba)
Estimated effort	57–79 developer-days

1. Two companies, one platform — the shared data question

What already exists

The backend models both companies, but shallowly. `CompanyType` is an enum with two values, and it appears in exactly one entity:

users	—	shared, no company column
reservations	—	shared, no company column
tours	—	shared, no company column
tour_types	—	shared, no company column
extras	—	shared, no company column
transactions	—	shared, no company column
invoices	—	company_type ← the ONLY place the distinction exists
document_sequences	—	UNIQUE(type, year) ← shared across BOTH entities

`InvoiceEmailService` then branches on it to select a logo, a stamp image and a legal footer:

COMPANYTYPE	FOOTER PRINTED ON THE INVOICE
DUNES_INSOLITES	EL FAOUAR 4264 KEBILI TUNISIE · Tel/Fax 75 461 016 · GSM 27 391 501 · dunesinsolites@gmail.com
ROUTE_INSOLITE	BOULEVARD 11 JANVIER, ROND-POINT FATOU, HOUMET SOUK DJERBA, 4180 · Tel +216 50 655 844 · contact@route-insolite.com

VERDICT: YES, SHARE THE PLATFORM — THE SHARED USER TABLE IS THE RIGHT CALL

These are not competitors. They are two entities of one operation: the agency in Djerba sells circuits that *overnight at the camp* in Sabria — your own `ReservationTourHebergement` entity models exactly that. Customers cross over. Staff overlap. A single platform with proper company scoping is correct, and splitting into two databases would be strictly worse: you would lose the ability to book a circuit-plus-camp-stay as one reservation, and you would need cross-system reconciliation to answer basic questions.

BUT THE CURRENT MODEL IS UNDER-BUILT. NINE ISSUES, IN SEVERITY ORDER.

MT-01 Two legal entities share one invoice number sequence

2d

`DocumentSequence.java · ReservationServiceImpl.generateFactureNumber():1774`

Why this is serious: `DocumentSequence` is unique on `(type, year)` only, and `generateFactureNumber()` looks up `("FACTURE", year)` with no company dimension. Both entities draw from the same counter.

The consequence in each entity's books:

```
// Both companies invoicing in 2026, one shared counter:
FACTURE 001/2026 → Dunes Insolites
FACTURE 002/2026 → Route Insolite
FACTURE 003/2026 → Route Insolite
FACTURE 004/2026 → Dunes Insolites

// Dunes Insolites' ledger: 001, 004, ... ← gaps at 002, 003
// Route Insolite's ledger: 002, 003, ... ← gaps at 001, 004
```

Tunisian invoicing requires each legal entity to keep its own sequential, gapless numbering — the same rule applies in essentially every jurisdiction. Neither entity's books survive an audit, and the missing numbers look like deleted invoices, which is what auditors specifically look for. The presence of `timbreFiscal` and a 7% `tvaRate` in `Invoice` confirms these are real fiscal documents, not internal receipts.

Fix: add `companyType` to `DocumentSequence`; change the constraint to `UNIQUE(type, year, company_type)`; key the lookup on all three. Add a unique constraint on `(invoice_number, company_type)` — `invoiceNumber` has none today. Prefix the number per entity (`DI-001/2026`, `RI-001/2026`) so a document is self-identifying.

Then deal with the existing data. Invoices already issued cannot be renumbered — they are in customers' hands and in the accounts. Work with the accountant on the correct remedy per entity; do not silently rewrite history in a migration.

Done when: each entity's factures form an unbroken sequence for the current year, and a concurrency test issuing from both companies at once produces no collision.

MT-02 `toggleCompanyType` rewrites an issued invoice's legal identity

1d

`InvoiceServiceImpl.toggleCompanyType():138 · InvoiceController:107`

Why: the endpoint flips an invoice between the two entities and saves. Nothing checks the invoice's status. A facture already numbered, stamped, PDF'd and emailed can be switched to the other company — keeping its number and date, but now carrying a different legal entity's name, address and stamp.

In accounting terms you cannot re-issue a document under a different entity. The correct remedy is a credit note against the first entity and a fresh invoice from the second.

Fix: permit the toggle only while `status = DRAFT` and no number has been assigned. Once issued, expose "cancel and re-issue under the other entity" instead, which produces a credit note and a new document. Add an audit record for every such action — who, when, why.

Done when: toggling a sent facture is refused with a clear error, and the re-issue path produces a credit note plus a correctly numbered new invoice.

MT-03 Company lives on the invoice, not the reservation

3d

Why: the company is only decided when an invoice is generated. A reservation that is never invoiced — cancelled, pending, paid in cash — belongs to no company at all. You cannot answer "what did Route Insolite sell in July?" without joining through invoices and losing everything uninvoiced.

Fix: put `companyType` on `Reservation`, set at creation from the product being booked, and default the invoice from it rather than asking again. `ReservationController` already accepts a `CompanyType` parameter in two places — make it a first-class column instead of a request-time argument.

Done when: every reservation, in any status, is attributable to exactly one company.

MT-04 The statistics module mixes both companies' money

2d

/api/admin/statistics/* · StatisticsRepository · StatisticsService

Why: none of the six statistics endpoints has a company dimension. The dashboard adds Dunes Insolites' and Route Insolite's revenue into one number. For two separate legal entities filing separate accounts, that figure is not meaningful to either.

Fix: add a company filter to every statistics query (depends on MT-03), and a company switcher in the admin dashboard with an "all companies" view for the group owner. Partner revenue and revenue distribution especially must be per-entity.

Done when: the dashboard reports each entity separately, and the two figures reconcile to the combined total.

MT-05 The product catalogue is not scoped by company

3d

Why: `Tour`, `TourType` and `Extra` have no company column, so the Djerba circuits and the Sabria nuitées are one undifferentiated list. Neither public site can filter to its own products, and the admin cannot either. This is the same problem the SEO Plan describes at the content layer — here it is at the data layer.

Fix: add `companyType` to the three catalogue entities. Model shared products explicitly if any genuinely belong to both — a nullable column meaning "both" is tempting and will cause confusion; prefer an explicit third state or a join table.

Done when: `GET /api/public/activities` returns only the calling site's products.

MT-06 Staff roles have no company dimension

3d

Why: roles are `ADMIN`, `CAMPING`, `PARTENAIRE`, `CLIENT` — no company. A `CAMPING` user at Sabria can read every Route Insolite reservation, and vice versa. For customers a shared account is correct and desirable; for staff it is a data-access problem between two legal entities.

Fix: keep one Keycloak realm — a realm per company would break single sign-on and double the administration for no gain — and add a company attribute to staff users, surfaced as a JWT claim. Enforce it in the repository layer, not only in the UI: a company-scoped query, not a hidden menu item. Group-level staff get an explicit "all companies" grant.

Done when: a Sabria `CAMPING` user requesting a Route Insolite reservation by id receives 403, verified by test.

MT-07 Partner commissions are per-user, not per-company

2d

Why: a `PARTENAIRE` carries one `commissionRate`. A travel agent working with both entities may well have different terms with each — different products, different margins, different contracts. One rate per user cannot express that. `UserProductRemise` is per-product and partially covers it; confirm whether it is actually used before adding another mechanism.

Fix: scope commission to `(user, company)` or rely entirely on `UserProductRemise` once products are company-scoped by MT-05. Pick one; do not run both.

Done when: a partner can hold different terms with each entity, and invoices apply the right one.

MT-08 Shared customer data across two legal entities

2d

Why: two separate legal entities sharing one `users` table means each is processing personal data collected by the other. A customer who booked a circuit from Route Insolite has not necessarily consented to the camp holding their details. With European guests this is a GDPR question with real exposure, and the vitrine already ships a `CookieConsent` component and privacy pages that currently describe one company.

Fix: document the lawful basis for sharing — most likely a joint-controller arrangement between the two entities, which must be written down and disclosed. Update both privacy policies to name both entities. Make sure deletion and export requests span both. This is a legal review, not only a code change.

Done when: both privacy policies name both entities and a deletion request removes the customer from both companies' records.

application.yml — app.mail.from • EmailService • InvoiceEmailService

Why: `app.mail.from` is fixed at `noreply@duneinsolite.com` and SMTP authenticates as a single Gmail account. A Route Insolite invoice therefore arrives from a Dunes Insolites address while displaying Route Insolite's letterhead and stamp. That is confusing to the customer and a deliverability risk — the sending domain will not align with the branding.

Fix: per-company sender and reply-to, driven by `CompanyType`, with SPF/DKIM configured for both domains. The same applies to `app.frontend.url`, which is used for links inside emails and currently points at one host.

Done when: a Route Insolite invoice arrives from a `route-insolite.com` address that passes SPF and DKIM.

Recommended target model

users	— shared + company_affinity (staff only; customers stay global)
reservations	— + company_type (MT-03)
tours / tour_types	— + company_type (MT-05)
extras	— + company_type (MT-05)
invoices	— company_type (exists)
document_sequences	— UNIQUE(type, year, company_type) (MT-01)
statistics queries	— company-filtered (MT-04)
JWT	— + company claim for staff (MT-06)

One database, one realm, one codebase. Company becomes a first-class dimension rather than an invoice-time afterthought. Total for MT-01...MT-09: **19 days**, and MT-01 and MT-02 should be treated as urgent regardless of the rest of the roadmap, because they affect documents that have already been issued.

2. Separating espace client from espace admin

You asked for the two to be separated. They should be — they are opposites in almost every dimension that matters, and mixing them creates problems that are hard to unwind later.

DIMENSION	ESPACE CLIENT (VITRINE)	ESPACE ADMIN (BACKOFFICE)
Audience	Anonymous public, indexed by Google	Authenticated staff only, <code>noindex</code>
Rendering	Static/ISR, cached hard, revalidate 300–3600s	Dynamic, uncached, always fresh
Auth	Mostly none; light customer area	Mandatory, role-gated on every route
Bundle priority	Every kilobyte affects Core Web Vitals and ranking	Bundle size barely matters; tables and charts are fine
Release cadence	Tied to marketing and the SEO migration	Tied to operations; ships whenever
Failure impact	Lost bookings, lost rankings	Staff fall back to the old tool
Domain	<code>www.dunes-insolites.com</code>	<code>admin.dunes-insolites.com</code>

Recommended layout

dunes-insolites/	
├ apps/	
│ └ vitrine/	Next.js – public site, FR default + /en
│ │	<code>www.dunes-insolites.com</code>
│ └ admin/	Next.js – backoffice, all roles
│	<code>admin.dunes-insolites.com</code>
├ packages/	
│ └ api-types/	shared DTO contract (from <code>lib/types.ts</code>)
│ └ api-client/	typed fetch layer + BFF auth helpers
│ │	
│ └ ui/	only genuinely shared primitives
└ backend/	Spring Boot (separate repo today)

Why a workspace monorepo rather than two repositories: the two apps must agree on the API contract, and the whole reason the current vitrine and backend diverged is that a shared contract lived in a Markdown file instead of in code. Put the types in a package both apps import, and a breaking backend change fails both builds at once instead of silently at runtime.

Why not route groups inside one Next.js app: tempting, and wrong. Admin code would ship in the public bundle, the caching models are opposites, and a single deployment means an admin bug can take the public site down during peak booking season.

Why: the vitrine asks for httpOnly cookies (correctly — a token in `localStorage` is readable by any injected script) while the backend returns bearer tokens in JSON, and the Angular `partner-app` and `camping-app` depend on that JSON. All four must coexist.

- Spring stays a pure stateless resource server. Its contract does not change, so the Angular apps keep working untouched.
- Each Next.js app runs its own BFF: login proxies to `POST /api/auth/login`, tokens go into httpOnly/Secure/SameSite cookies scoped to that app's own subdomain, and server-side code attaches `Authorization: Bearer` when calling Spring.
- **Separate cookie scopes.** The admin cookie must be scoped to `admin.dunes-insolites.com` and never to the parent domain — otherwise a vulnerability on the public marketing site reaches an admin session.
- Roles come from `realm_access.roles`, read server-side, used for navigation only. Every decision is re-checked by Spring.
- Refresh happens server-side against `POST /api/auth/refresh`.

Done when: both apps authenticate with no token in `document.cookie` or `localStorage`, admin cookies do not leak to the apex domain, and the Angular apps are unaffected.

`SecurityConfig.bearerTokenResolver()` · `NotificationController`

Why: the backend added an `?access_token=` query-parameter fallback for `/api/notifications/subscribe`, because the browser `EventSource` API cannot set headers. Reasonable and RFC-6750-documented — but it only works if the browser holds the token, which is exactly what the cookie decision forbids.

Fix: proxy the stream. A Next.js route handler opens the SSE connection to Spring server-side with the bearer header and re-streams it to the browser over the session cookie. The token never leaves the server, the query-parameter fallback stays unused in production, and `EventSource` works normally client-side.

Done when: live notifications arrive in the admin with no access token anywhere in the browser.

Detail for all 27 defects — file, line, reasoning, acceptance criteria — is in the Production Readiness Plan. Summarised here with the changes this document introduces.

#	PHASE	DAYS	CONTENT
0	Stop the bleeding	1	Role escalation via public registration , committed secrets, CORS configured but never applied, five unprotected controllers
1	Make the API usable	4–5	Exception→status mapping, bean validation, atomic Keycloak registration, DTO leakage, ARCH-01 session model, CSRF
1b	Company scoping	19	MT-01...MT-09 from section 1 — new
2	Public API	10–13	Public read layer, availability, bookings, contact/newsletter, kill silent seed fallback · + legacy French slugs and FR/EN content
3	Data & correctness	5–7	Flyway, authorization-matrix tests, unbounded queries, money handling
4	Production hardening	5–7	Config/profiles/TLS, rate limiting, observability, CI/CD

A-01 Phase 0 — four live exploits, one day

9h

- **Role escalation.** `POST /api/auth/register` is `permitAll()` and passes the body's `role` into `assignRole()`. Anyone can `POST {"role":"ADMIN"}` and receive a realm admin account. Delete the field; hard-code CLIENT.
- **Committed secrets.** `.env` is tracked in git — while its own first line claims it is gitignored. Rotate the Gmail app password and Keycloak client secret first, then untrack.
- **CORS never applied.** `CorsConfig` declares the bean; `SecurityConfig` never calls `.cors(...)`. The bean is inert and every browser cross-origin request is rejected.
- **Five open controllers.** `Chauffeur`, `Guide`, `Source`, `Notification` and `CampingSettings` have no `@PreAuthorize`, so any logged-in CLIENT can delete guides and booking sources.

Done when: each is verified by request, not by reading code.

A-02 Public API must carry legacy French slugs and both locales

+3d

Why: the SEO Plan requires the public route key to be *the exact legacy WordPress slug*, and every public entity to carry French and English content. This is the dependency between the two documents.

- Add `slug` as a unique column on `Tour`, `TourType` and `Extra`, seeded from the legacy slugs in the SEO Plan's URL map.
- Add a translation table keyed on (entity, locale, field) for the translatable fields — `name`, `description`, `aboutText`, `highlights`, `includedItems`, `programSteps.*`. Dedicated `*_fr` / `*_en` columns are quicker now and much worse later.
- Public endpoints accept `?locale=fr|en` and fall back to French.
- New package `controller.publicapi` serving `/api/public/**: permitAll()`, read-only, cacheable, with **dedicated DTOs** — never reuse admin DTOs, whose cost prices, commission rates and partner data must not reach a public page.
- `Tour` already denormalises `averageRating` and `reviewCount`; use those for listings and keep them updated on review approval.

Done when: `GET /api/public/stays?locale=fr` returns French content under the legacy slugs.

[Invoice.java](#) · [Tour.java](#) · [TourType.java](#)

Why: `totalAmount`, `totalHt`, `tvaAmount`, `timbreFiscal`, `totalTtc` and every price on `Tour` / `TourType` are `Double`.

Binary floating point cannot represent decimal currency exactly. On documents carrying a 7% TVA calculation and a fiscal stamp, rounding drift is not cosmetic — it is a figure that will not reconcile.

Fix: migrate all monetary fields to `BigDecimal` with explicit scale and `RoundingMode`. Note `InvoiceEmailService` already formats to three decimals (`%.3f`) and rounds via `Math.round(v * 1000.0) / 1000.0`, which is the millime — correct for Tunisian dinars, but the storage type undermines it. Store the quoted currency alongside every amount.

Done when: displayed price, stored price and invoice line agree to the millime in every supported currency.

A Next.js admin replacing the Angular `admin-app`. The backend already exposes nearly everything it needs, so this is largely a UI build against an existing API — but a booking backoffice for two legal entities is a genuinely large surface.

BEFORE WRITING ANY OF IT: GET READ ACCESS TO ROUTEINSOLITE

The Angular `admin-app` is deployed, in production use, and is the de-facto specification for what the business actually does daily. It is not on this machine — it was authored by `bacem.benakkari@polytechnicien.tn`. Even if not one line is reused, it tells you which of the 19 controllers are genuinely exercised and which screens matter. Rebuilding blind guarantees shipping the wrong 20% and discovering it at handover.

B-00 App skeleton, auth shell, shared packages

4d

- Monorepo per section 2; `packages/api-types` becomes the contract both apps import.
- BFF auth per ARCH-01; middleware guards every route; navigation driven by role *and company* (MT-06).
- **Company switcher in the shell** — most screens are company-scoped, and this should be a global control, not a filter bolted onto each table.
- Shared primitives: data table (TanStack Table — sorting, server pagination, column filters), forms (react-hook-form + zod schemas derived from shared types), error surface wired to Phase 1 status codes, confirmation dialogs on destructive actions.
- Dynamic rendering by default. None of the vitrine's ISR patterns apply.

Done when: an ADMIN logs in, a CLIENT is refused, the company switcher filters a list, and a shared type change breaks both builds.

Modules

ID	MODULE	BACKEND ENDPOINTS (ALREADY EXIST)	ROLES	EST.
B-01	Dashboard	<code>/api/admin/statistics/*</code> — 6 endpoints (+ <i>company filter</i> , MT-04)	ADMIN	3d
B-02	Reservations — core	<code>/api/reservations</code> + search, by-date, filter, status, active, staff assignment, generate-facture	A/C/P	7d
B-03	Catalogue — tours	<code>/api/tours</code> CRUD + deactivate (+ <i>slug & locales</i> , A-02)	ADMIN	3d
B-04	Catalogue — types & extras	<code>/api/tour-types</code> , <code>/api/extras</code>	A/C	2d
B-05	Users & discounts	<code>/api/users</code> , <code>UserProductRemise</code>	ADMIN	3d
B-06	Staff	<code>/api/guides</code> , <code>/api/chauffeurs</code>	ADMIN	2d
B-07	Invoices	<code>/api/invoices</code> , factures, send-proforma, send-facture (+ MT-01, MT-02)	A/C	3d
B-08	Payments & transactions	<code>/api/transactions</code> , <code>/api/reservations/{id}/payments</code>	A/C	2d
B-09	Review moderation	<code>/api/reviews</code> — <i>needs an approval flag; the vitrine publishes these</i>	ADMIN	2d
B-10	Notifications (live)	<code>/api/notifications</code> + SSE (via ARCH-02 proxy)	all	2d
B-11	Settings & sources	<code>/api/camping-settings</code> , <code>/api/sources</code> , <code>/api/currency/rates</code>	A/C	2d

Modules subtotal	31d
------------------	-----

B-12 Hardening, audit trail and cutover

4d

- Every screen's authorization re-verified server-side, per role *and* company. Navigation hides what a role cannot do; the server refuses it regardless.
- **Audit log** on destructive and financial actions — who cancelled, who changed a price, who issued or re-issued a facture. The backend has none today, and MT-02 makes it necessary.
- Optimistic concurrency on reservation edits so two operators cannot silently overwrite each other.
- Cutover: run alongside `admin-app`, migrate users, then remove the Angular container from `docker-compose.yml` and drop its CORS origin.

Done when: the admin team works a full week without reopening the old tool.

Espace client — remaining work

The vitrine is largely built. What it still needs, beyond the SEO Plan:

Customer area	<code>/login</code> , <code>/signup</code> , <code>/bookings/[id]</code> exist as shells. Wire to the BFF; add booking history to replace WooCommerce's <code>/mes-reservations/</code> . 2d
Booking lookup security	<code>/bookings/{id}</code> uses a 6-character <code>DI-</code> reference — guessable. Require reference + email, or an opaque token. 1d
Kill the silent seed fallback	<code>lib/api.ts</code> returns seed data on any fetch failure. In production that serves stale marketing prices with no error anywhere. Gate on <code>NODE_ENV</code> ; fail the build if the API URL is unset in prod. 1d
Real availability & bookings	Availability is currently a hash of the date string; bookings live in an in-memory <code>Map</code> . Both covered in A Phase 2. 5d

5. Sequence, effort and decisions

Dependencies

1. **A Phase 0 blocks everything.** One day. Nothing else starts while an anonymous user can mint an admin account.
2. **MT-01 and MT-02 are urgent independently of the roadmap** — they concern documents already issued to customers and already in the accounts. Do them early even if the rest of section 1 waits.
3. **The SEO Plan's SEO-01 and SEO-03 must be decided before A Phase 2 is built**, because they determine the slugs and locales the public API serves. Building the API first and retrofitting means building it twice.
4. **B needs A Phase 1** (the session model) but not Phase 2, so the backoffice runs in parallel with the public API work.

WEEKS	TRACK 1	TRACK 2 (SECOND DEVELOPER)
Week 1	A Phase 0 (1d) → MT-01, MT-02 → A Phase 1	SEO-01 decision · SEO-02 · baseline archive
Weeks 2–3	MT-03...MT-06 company scoping	B-00 foundations · B-01 dashboard
Weeks 4–6	A Phase 2 — public API, slugs, locales	B-02 reservations · B-03/B-04 catalogue
Weeks 7–8	SEO-04 i18n · SEO-05 blog	B-05...B-08 users, staff, invoices, payments
Weeks 9–10	A Phase 3 — Flyway, tests, A-03 money	B-09...B-11 · B-12 hardening
Week 11	A Phase 4 — config, rate limiting, CI	Admin cutover, retire <code>admin-app</code>
Week 12	SEO-09 cutover — then 30 days of monitoring	

Effort

WORKSTREAM	DAYS	DOCUMENT
A — Backend hardening & public API	27–37	this + Production Readiness Plan
↳ of which company scoping (MT-01...09)	19	this, section 1
B — Espace admin	25–35	this, section 4
Espace client — remaining	5–7	this, section 4
Full-stack total	57–79	
C — SEO & migration	18–24	SEO Plan
Programme total	75–103	15–21 weeks solo · 9–12 weeks with two

Excludes content translation and copywriting, photography, WooCommerce customer-data migration (cannot be sized until the data is inspected), and any accountant time needed for MT-01's historical remedy.

DECISIONS NEEDED BEFORE WEEK 2

1. **Access to `routeinsolite`**. The Angular admin-app is the specification for Workstream B.
2. **Fate of `partner-app` and `camping-app`**. If they stay Angular you are committing to two frontend stacks permanently. This changes whether B-00 is built as an ADMIN app or a role-aware shell.
3. **MT-01 historical remedy**. Needs your accountant, not a developer. What happens to invoices already issued from a shared sequence?
4. **MT-08 data-sharing basis**. Needs legal input on the joint-controller arrangement between the two entities.
5. **Domain consolidation**. The backend infrastructure lives on `dunesinsolites.com` subdomains while the public site is `dunes-insolites.com`. Recommend consolidating onto the hyphenated domain and 301ing the rest.
6. **WordPress booking work**. Pages dated July–August 2026 suggest a booking flow is being built inside WooCommerce in parallel. Confirm it is stopped.

The three things most likely to go wrong

RISK	SEV	MITIGATION
Invoice numbering discovered by an audit before it is fixed	CRITICAL	MT-01 and MT-02 in week 1; involve the accountant on the historical remedy immediately
Backoffice rebuild misses what the business actually uses	HIGH	Get routeinsolite access; walk the admin team through a real week before designing screens
Scope grows because three systems are merging at once	HIGH	Hard gates between workstreams; no new features during a replatform

Prepared 25 August 2026. Sources: static analysis of the Next.js vitrine (d4c404e) and Spring Boot backend (ae970bc), docker-compose.yml , and live checks of www.dunes-insolites.com and www.route-insolite.com. Companion document: the SEO Plan, covering brand territory, URL migration, i18n and Core Web Vitals. The Production Readiness Plan remains valid as the line-level detail for Workstream A.